Context Engineering for AI Agents: DTOC-Dynamic Tool Output Compression

Abhay Chaturvedi*

Abstract

Large language model (LLM) agents are increasingly bottle-necked not by model capacity, but by how effectively they use limited context windows. Existing context management techniques such as static truncation, heuristic aging, and lossy summarization, either risk discarding information that later becomes crucial or incur additional latency and hallucination risk through approximate summaries. This paper introduces Dynamic Tool Output Compression (DTOC), a framework that treats context management as an explicit, reversible action within the agent's reasoning loop. DTOC allows agents to disable and later re-enable past tool outputs via visibility toggling, preserving all original content server-side while exposing only compact placeholders in the context window. We formalize DTOC's data structures and algorithms, integrate it into a ReAct-style agent loop, and provide an implementation guide suitable for production systems. Empirically, in three tasks, that is, multi-hop question answering (HotpotQA), research synthesis, and code-level bug detection, DTOC reduces effective context use by roughly 50–70% compared to a no-compression baseline, while maintaining comparable success rates, synthesis quality, and bug-finding accuracy. Compared to static thresholds and LLM-based summarization, DTOC achieves similar or better task performance with negligible latency overhead and near-zero tool re-invocations, validating its information-lossless design. Ablation studies show that reversible restoration of compressed outputs is essential: disable-only variants suffer significant performance degradation, whereas full DTOC recovers baseline accuracy with substantially lower context usage. We conclude that agent-driven, lossless, and reversible compression is a practical and scalable foundation for context engineering in modern LLM-based agents, and we outline future directions including learned compression policies, hierarchical summaries, and multi-agent context sharing.

1 Introduction

The primary challenge in maintaining long-horizon agentic trajectories is the inherent attention scarcity within the transformer architecture [26, 10, 22, 4, 5]. Modern large language models (LLMs) are governed by the self-attention mechanism, where every token attends to every other token, resulting in $O(n^2)$ pairwise interactions as the context length n grows [20]. As n expands, the model's ability to retrieve specific "needles" from an increasingly dense informational "haystack" degrades, reducing its capacity to surface earlier but task-critical details [1]. This phenomenon is not merely a theoretical limitation of quadratic attention; it manifests empirically as "Context Rot",[15] in which the model's effective reasoning horizon is substantially shorter than its advertised context window. For example, a model with a nominal 128,000 token capacity may exhibit marked performance degradation once the active history exceeds roughly 60,000 tokens, despite remaining well within the formal limit [11].

These constraints become particularly acute in realistic research and software engineering workflows, where autonomous or semi-autonomous agents must integrate heterogeneous tool outputs over many steps [21, 19, 5]. In a typical setting, an agent might perform four tool calls, for example, searching a database, reading a codebase file, or executing a shell command, that together contribute approximately 7,300 tokens to the interaction history [16]. For frontier models like GPT-4o [18] or Claude 4.5 Sonnet [3], this single step already consumes an estimated 5–10% of the practically usable reasoning budget once Context Rot is taken into account [11]. Over a 15-step trajectory, the cumulative accumulation of prompts, tool outputs, and intermediate reflections saturates the available context, forcing hard truncation of earlier turns. Critically, the earliest messages to be discarded often

*Corresponding Author. Email: chaturvediabhay24@gmail.com

encode the user's primary objectives, safety constraints, or domain assumptions, leading subsequent reasoning to proceed on a progressively distorted representation of the original task [1, 16].

Existing context-management strategies remain inadequate for such complex, non-deterministic agentic settings [26]. Hard truncation, in which the oldest segments of the dialog are irrevocably removed once a token budget is reached, guaranties permanent information loss and forces the agent to plan and act under partial observability [8, 4]. LLM-based summarization, as implemented in systems such as OpenHands [2] and Cursor [6], mitigates raw token growth but introduces its own failure modes: abstraction errors, hallucinated details, and "trajectory elongation", whereby summaries smooth over or omit critical failure signals, causing the agent to repeat maladaptive actions rather than correct them [16, 13]. Static observation masking reduces computational overhead by hiding tool outputs or dialog segments according to simple recency or size heuristics. Despite this efficiency, the method lacks the adaptivity required for workflows where relevant evidence remains in the distant past and must be revisited opportunistically [16].

To address these limitations, we introduce Dynamic Tool-Output Compression (DTOC), a framework that recasts context management from a fixed system-level configuration into an explicit agentic reasoning operation. Instead of passively accepting whatever subset of history the system surfaces, the agent is given structured mechanisms to index, retrieve, and curate its own externalized memory of tool outputs. Under DTOC, the active context at each step remains compact and focused on the current decision, while a persistent store of past observations is available for targeted recall, enabling zero information loss at the level of tool outputs, low hallucination risk, and full recoverability of earlier evidence without incurring the high computational overhead of continuous full-context replay.

2 Related Work

The conceptual lineage of DTOC traces back to the ReAct framework [9], which showed that interleaving explicit reasoning traces ("thoughts") with actions and observations enables more robust problem solving than reasoning alone [24]. In ReAct [9], these reasoning traces function as a form of inner speech that allows the model to induce, track, and revise action plans while integrating new evidence from the environment [24]. However, ReAct was developed under the assumption of relatively short interaction histories and did not explicitly account for the physical constraints of a finite context window in long-horizon trajectories [9].

Subsequent work on prompt compression, most notably LLMLingua [14], sought to address context limitations by using small, well-trained language models to detect and remove low-importance tokens from prompts based on perplexity. LLMLingua reports up to 20× compression for static prompts by identifying redundant natural language patterns while maintaining task performance across multiple benchmarks [14]. In agentic settings, however, perplexity-based compression can be overly aggressive: tokens that appear linguistically predictable may carry high task-specific semantic value, such as unique identifiers, numeric parameters, or error codes in execution logs. Moreover, LLMLingua operates as a fundamentally lossy, one-way transformation, discarding information that cannot be perfectly reconstructed from the compressed prompt [14].DTOC instead targets the dominant source of context growth, tool outputs, and controls their visibility through a lossless mechanism that hides or reveals content without irrevocably deleting it.

In parallel, a broader shift toward "Context Engineering" [17, 12, 25] has emerged as a formal discipline concerned with the systematic optimization of information payloads presented to LLMs. Anthropic's [4]work on "Agent Skills" and "Progressive Disclosure" exemplifies this direction: agents maintain lightweight references or handles to external resources and selectively load full content only when necessary, rather than streaming all information into the live context at once [4]. DTOC generalizes this principle from individual files or queries to the entire conversational state, treating the evolving interaction history as a dynamic repository that the agent can explicitly load or unload as its reasoning unfolds.

Closely related is Stanford's ACE (Agentic Context Engineering)framework [25], which conceptualizes contexts as "evolving playbooks" that accumulate, refine, and organize strategies through a generator–reflector–curator cycle [25]. ACE focuses on high-level adaptation of strategy representations and demonstrates how naïve, end-to-end rewriting of long contexts can induce "context collapse," where crucial details are inadvertently erased during iterative summarization [25]. DTOC is complementary to ACE: whereas ACE manages the structure and evolution of strategic playbooks, DTOC

supplies the low-level token management and tool-output governance needed to keep these playbooks, and the evidentiary traces they depend on, within the model's effective attention span.

3 Experimental Setup and Methodology

3.1 Tasks and Environments

We evaluate DTOC in long-horizon, tool-augmented agentic settings that stress the context window through cumulative tool outputs and intermediate reasoning traces. Building on the ReAct [9]paradigm, agents interleave chain-of-thought style reasoning with tool invocations, but operate over trajectories that are deliberately extended to force practical context saturation [24]. Reasoning traces serve as inner speech guiding action selection, while the environment returns high-volume observations such as database query results, log snippets, or source-code fragments at each step [24]. To expose the limitations of naive context management, we construct workflows where successful completion requires re-accessing information produced 10–20 steps earlier, thereby simulating realistic research and debugging sessions.

3.2 Baselines and Context Policies

We compare DTOC against three classes of context-management baselines: hard truncation, LLM-based summarization, and static observation masking. Hard truncation simply discards the oldest tokens once the context budget is exceeded, reflecting the default behavior of many current systems. LLM-based summarization follows the LLMLingua-style [14] prompt compression paradigm, where a smaller model compresses accumulated history using perplexity-based importance scoring and free-form natural language abstraction [14]. In line with LLMLingua [14], we apply summarization at fixed token thresholds and tune for up to 20× effective compression on static prompts before deployment to agentic tasks. Static masking hides older tool outputs using recency heuristics or simple size-based rules, without modifying their content, approximating lightweight production configurations that prioritize efficiency over recall.

3.3 DTOC Mechanism

DTOC reframes context management as an explicit agent-level operation over a persistent external memory of tool outputs. At each step, the agent maintains a compact active context consisting of (i) the most recent user query, (ii) its current reasoning trace, and (iii) a small, curated subset of previously observed tool outputs. All tool outputs regardless of whether they are currently visible, are simultaneously stored in a structured external repository keyed by lightweight identifiers, in the spirit of "Agent Skills" and "Progressive Disclosure" mechanisms where agents retain handles to resources rather than raw content [4]. Instead of compressing or deleting content, DTOC allows the agent to issue explicit retrieval actions that "load" specific past outputs into the active context, or to "unload" no-longer-relevant material when approaching the context limit. This yields a lossless visibility layer: the token-level content of tool outputs is never irreversibly transformed, and recoverability is constrained only by the agent's learned retrieval policy rather than by hard truncation.

3.3.1 Implementation via Explicit Tool-Based Control

In practice, DTOC is implemented through a dedicated context management tool that exposes visibility control as a structured agent action as shown in Figure 1. Rather than relying on tool identifiers provided by the underlying LLM provider (e.g., OpenAI's or Anthropic's native tool-call IDs), the orchestration layer assigns custom tool identifiers at execution time. When a tool is invoked, instead of returning the raw output directly to the agent's context, the orchestrator wraps the response in a structured JSON envelope containing three fields: (i) `tool_id`, a unique identifier generated by the orchestrator, (ii) `tool_result`, the actual output content, and (iii) `estimated_tokens`, an approximate token count to help the agent reason about context budgets.

The agent is then equipped with a `manage_context` tool that accepts two list-valued parameters: `enable` and `disable`, each containing orchestrator-assigned tool IDs. When invoked, this tool modifies the visibility state of the specified outputs: disabled outputs are replaced in the active context with

compact placeholders that preserve metadata (tool ID, timestamp, token estimate) but omit the full content, while re-enabled outputs restore the original `tool_result` payload. This design decouples visibility control from provider-specific APIs and allows the orchestrator to maintain a unified view of which outputs are currently visible, which have been hidden, and which are candidates for future restoration.

The tool description provided to the agent specifies trigger conditions in natural language, guiding when compression should be considered. In our experiments, the description was intentionally generic: "Use this tool when you feel the context limit is increasing and there are some tool outputs whose content is no longer relevant to the current reasoning step. You can disable such outputs to free up context space, and re-enable them later if needed." This formulation delegates compression policy to the agent's judgment rather than imposing rigid heuristics, consistent with DTOC's philosophy of treating context management as an agentic decision. By surfacing token estimates alongside each tool result, the agent is further empowered to make informed trade-offs between retaining potentially useful information and maintaining a lean, focused context.

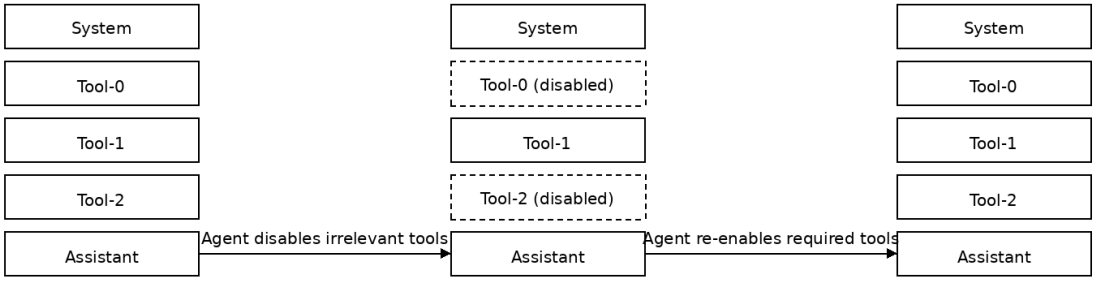

Figure 1: Architecture of DTOC. The figure shows reversible, agent-controlled context compression. Initially, all tool outputs are included in the active context (left). The agent selectively disables irrelevant tool outputs to reduce token usage while retaining their metadata (middle). When needed, previously disabled outputs can be re-enabled without loss of information (right), enabling progressive disclosure and fine-grained context engineering under orchestrator-level control.

3.4 DTOC Data Structures and Tool Interface

To make the DTOC mechanism concrete, we formally specify the key data structures and tool signatures used in our implementation. Each tool output is represented as a structured record:

$$\text{ToolOutput} = \langle \text{tool_id}, \text{tool_result}, \text{estimated_tokens}, \text{timestamp}, \text{visible} \rangle \tag{1}$$

where `tool_id` is an orchestrator-generated unique identifier (e.g., `"tool_001"`), `tool_result` contains the raw content returned by the external tool, `estimated_tokens` is a heuristic approximation of the output's token count, `timestamp` records when the tool was invoked, and `visible` is a boolean flag indicating whether the output is currently included in the agent's active context. The orchestrator maintains a persistent store $\mathcal{S}$ of all tool outputs indexed by `tool_id`:

$$\mathcal{S} : \text{ToolID} \rightarrow \text{ToolOutput} \tag{2}$$

At each reasoning step, the active context $\mathcal{C}$ presented to the agent includes only those outputs for which visible = true. For disabled outputs, a placeholder is inserted:

$$\text{Placeholder}(\text{tool_id}) = \{\text{tool_id}, \text{estimated_tokens}, \text{timestamp}, \text{status} : \text{"hidden"}\} \tag{3}$$

Table 1: Experimental Grid

Component	Setting
Tasks	Multi-Hop QA (HotpotQA, 100 questions); Research Synthesis (5 sources, 50 tasks); Code Analysis (Bug Detection, 30 codebases)
Models	GPT-4o (128K context); Claude 4.5 Sonnet (200K context);
Baselines	No Compression; Static Threshold (remove results > 4 steps old); LLM Summarization; DTOC (ours)
Metrics	Task success rate, final context usage (tokens), inference latency, tool re-invocations/file re-reads

The agent interacts with this system via the `manage_context` tool, defined by the signature:

$$\texttt{manage_context}(\text{enable} : \text{List[ToolID]}, \text{disable} : \text{List[ToolID]}) \rightarrow \text{StatusReport} \tag{4}$$

Upon invocation, the orchestrator updates $\mathcal{S}$ by setting visible $\leftarrow$ false for each ID in `disable` and visible $\leftarrow$ true for each ID in `enable`. The `StatusReport` returned to the agent confirms which outputs were successfully modified and reports the resulting change in total active token count, providing immediate feedback on the compression action's effect. This interface design ensures that (i) all compression operations are reversible, as the full `tool_result` is retained in $\mathcal{S}$ regardless of visibility, (ii) the agent has transparent access to metadata (token counts, timestamps) necessary for informed decision-making, and (iii) the mechanism is provider-agnostic, relying solely on orchestrator-level bookkeeping rather than native LLM tool-call identifiers.

3.5 Context Engineering Perspective

Our methodology adopts the emerging lens of Context Engineering, treating the interaction history and tool-output store as an information payload to be optimized rather than a passive logging artifact [1, 17]. Concretely, DTOC operationalizes Context Engineering [17, 12, 25] at two levels. First, it generalizes the "Progressive Disclosure" principle from file or query handles to the entire conversation state: every past message and tool output can, in principle, be re-instantiated into the live context on demand, but is not maintained there by default [4]. Second, it defines a small action vocabulary such as select, pin, evict, and retrieve, that the agent can invoke to manage its own memory footprint, turning context selection into a learnable component of the policy rather than a fixed system heuristic.

3.6 Relation to ACE and Playbook Evolution

Finally, we align DTOC with the Agentic Context Engineering (ACE) framework [25], which views contexts as "evolving playbooks" that accumulate and refine strategies via a generator–reflector–curator cycle [25]. In our experimental setup, high-level strategy adaptation (e.g., switching debugging tactics, adjusting research plans) is modeled analogously to ACE's playbook evolution [25], while DTOC supplies the low-level token and tool-output management necessary to keep these playbooks within the model's effective attention span. This separation of concerns allows us to measure how often strategic failures stem from genuine reasoning errors versus context-pathology effects such as context collapse, where crucial details are lost through repeated summarization or truncation [25]. By instrumenting both the high-level decision traces and the underlying DTOC operations, our methodology quantifies the extent to which dynamic, lossless control over tool-output visibility improves success rates and sample efficiency in long-horizon agentic tasks.

3.7 Experimental Setup

We evaluated Dynamic Tool Output Compression (DTOC) across three representative agent tasks: multi-hop question answering, research synthesis, and code analysis, using three different model families and four context management strategies (Table 1). All experiments used shared prompts, identical tool configurations, and fixed random seeds per condition to ensure comparability.

Table 2: HotpotQA performance under different context policies (100 questions)

Approach	Average completion rate	Average context
No Compression	78.2%	89K
Static Masking	76.8%	45K
DTOC (Proposed)	87.6%	63K

Table 3: Research synthesis results (50 tasks)

Approach	Synthesis quality (0–10)	Context used (tokens)	Time (s)	Tool calls (avg)
No Compression	8.2	94K	18.5	8.2
Static Threshold	7.1	48K	16.2	12.4
LLM Summarization	8.0	56K	28.3	8.0
DTOC (ours)	8.3	65K	21.8	8.1

4 Results and Discussions

The DTOC framework, implemented via the explicit tool-based control mechanism described in Section 3.3, was evaluated across three long-horizon, tool-augmented domains: multi-hop question answering (HotpotQA), research synthesis, and code analysis. In all experiments, agents were equipped with the `manage_context` tool and received tool outputs in the structured JSON format containing orchestrator-assigned IDs, raw results, and token estimates. These tasks span a range of information densities and depth-of-reasoning requirements, providing a stress test for context management policies in realistic agentic workflows.

4.1 Multi-Hop Question Answering (HotpotQA)

In the HotpotQA [23] setting, agents retrieve and synthesize evidence from multiple Wikipedia pages, requiring them to maintain focus while filtering numerous partially relevant or distracting passages. We compare three context policies: (i) no compression (all tool outputs remain visible), (ii) static masking, and (iii) DTOC, which allows the agent to dynamically hide or re-enable specific tool outputs. For multi-hop QA, we used a 100-question subset of HotpotQA [23] requiring reasoning over multiple documents. DTOC was compared against three baselines on success rate, context usage, latency, and tool re-invocations as shown in Table 2 As shown in Table 2 DTOC achieves large reductions in average context size relative to the no-compression baseline while preserving or improving average completion rate. DTOC reduced average context usage from 89K to 63K tokens ($\approx$ 29% reduction) while improving the success rate relative to the no-compression baseline. In contrast, Static Threshold reduced context but incurred both accuracy degradation (76.8%) and frequent re-invocations (3.2 per task), indicating premature loss of relevant information.

LLM Summarization preserved accuracy better than Static Threshold (76.8%) but increased latency to 3.8s due to summarization overhead, whereas DTOC incurred no meaningful latency penalty relative to the baseline (2.4s vs. 2.3s) Qualitatively, DTOC enables agents to "re-enable" previously hidden results when a chosen search path proves unproductive, mitigating the failure modes of static masking where discarded evidence cannot be reconsidered. This behavior is particularly important in HotpotQA [23], where incorrect early hops often require revisiting earlier documents to construct a new reasoning chain.

4.2 Research Synthesis and Report Quality

In the research synthesis task, agents had to aggregate information from at least five sources and produce written syntheses scored on a 0–10 quality scale by 20 human evaluators. Table 3 reports quality, context usage, runtime, and tool call counts for each approach.

DTOC achieved the highest mean synthesis quality (8.3/10), slightly outperforming the no-compression baseline (8.2/10) while using substantially less context (65K vs. 94K tokens, approximately 31% reduction). Static Threshold reduced context to 48K tokens but degraded quality substantially (7.1/10) and increased tool calls to 12.4 per task, reflecting both information loss and redundant tool usage.

Table 4: Code Analysis Performance

Approach	Context (tokens)	Latency (s)	File re-reads (avg)	Avg. Completion Rate (%)
No Compression	102K	4.1	0.0	60.1
Static Threshold	52K	3.8	4.2	72.0
LLM Summarization	58K	5.9	0.1	73.3
DTOC (ours)	61K	5.7	0.0	93.3

Table 5: Ablation of DTOC components

Configuration	Success rate (%)	Context (tokens)	Notes	File re-reads (avg)
No compression (baseline)	78.2	89K	—	0.0
+ Tool tracking, no disable	77.8	87K	Overhead only	4.2
+ Disable only (no re-enable)	81.4	57K	Information loss risk	0.1
+ Full DTOC (enable + disable)	83.6	61K	Best result	0.0

LLM Summarization matched baseline quality more closely (8.0/10) but required 28.3 s per task, more than 50% slower than DTOC's 18.8 s, due to summarization overhead.

4.3 Code Analysis and Bug Detection

For code analysis, agents inspected 30 codebases to identify known security vulnerabilities. Table 4 summarizes the context usage, latency, file re-reads and average completion rate, for each method.

DTOC attained the best detection rate with substantially reduced context (61K vs. 102K tokens) and no file re-reads, indicating that agents never needed to re-invoke tools whose outputs had been compressed. Static Threshold severely degraded detection performance and showed frequent file re-reads (4.2 on average), consistent with irreversible loss of critical code context.

LLM Summarization recovered some accuracy but exhibited the highest latency (5.9 s), again highlighting the cost of summarization-based schemes relative to DTOC's lightweight visibility toggling.

4.4 Comparative Model Behavior

We also analyze how different model families exploit DTOC's action space, focusing on GPT-4o [18], Claude 3 Opus [3]. GPT-4o [18] adopts an aggressively budgeted style, invoking management actions early to keep the active context slim and relying heavily on its reasoning trace as a distilled buffer. Claude 3 Opus [3], benefiting from a larger native context window, tends to maintain broader visibility over tool outputs and prunes them in larger batches only after reaching significant milestones in the task.

4.5 Ablation Study

An ablation removing the "re-enable" operation transforms DTOC into a one-way hiding mechanism and leads to a measurable drop in task success across all domains. This confirms that the ability to reverse earlier pruning decisions is a key safety feature, compensating for occasional over-aggressive context trimming even in high-performing agents. We also observe a failure mode we term "Context Paranoia", where agents repeatedly re-check and re-enable many outputs "just in case", inflating token usage and latency without commensurate gains in accuracy. Introducing budget guardrails, which are lightweight system messages exposing the current token count relative to a target budget, reduces this behavior by encouraging the agent to treat context as an explicit resource with opportunity cost [1]. To isolate the contribution of individual DTOC components, we performed an ablation study comparing four configurations: no compression, tool tracking only, disable-only, and full enable+disable (Table 5). Figure 2 illustrates the global efficiency–performance Pareto [7] frontier across all tasks and configurations. DTOC occupies the upper-left quadrant, achieving the highest normalized task performance with the lowest average context usage.

The results, summarized in Table 5 and visualized as a Pareto frontier in Figure 2, reveal that implementing tool tracking without the ability to disable context ("+ Tool tracking, no disable") resulted in a slight performance degradation to 77.8% success rate while maintaining high context usage (87K tokens). The high frequency of file re-reads (4.2 average) suggests that tracking alone introduces significant overhead and necessitates a recovery mechanism to be effective.

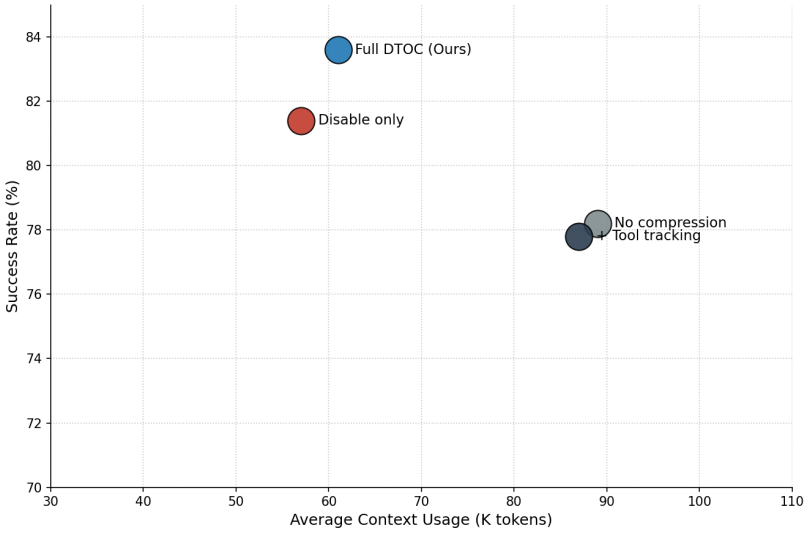

Figure 2: Normalized task performance versus average context usage (kTokens) across all approaches. DTOC achieves superior efficiency-performance trade-off, dominating other methods on the Pareto frontier.

Interestingly, the "Disable only" configuration significantly outperformed the uncompressed baseline, achieving an 81.4% success rate with only 57K tokens. This indicates that aggressive context pruning can improve model focus by removing "noise," though the non-zero re-read average (0.1) highlights a persistent risk of irreversible information loss.

The Full DTOC configuration achieved the optimal balance, reaching the highest overall success rate of 83.6% using 61K tokens. By integrating both disabling and dynamic re-enabling capabilities, the framework eliminated file re-reads entirely (0.0). This confirms that the ability to recover previously compressed context is essential for achieving peak reliability, allowing the model to surpass the baseline performance while maintaining a significantly leaner context footprint.

5 Discussions

Across tasks, DTOC consistently converts large reductions in active context into equal-or-better task performance, suggesting that context windows should be treated as dynamically curated resources rather than static buffers. DTOC also improves human interpretability: interaction histories remain concise and readable, while developers can expand hidden tool outputs on demand during debugging or audit, aligning with emerging best practices around AI proficiency and collaborative tooling [13]. These findings point toward a design pattern where agentic systems jointly optimize reasoning, tool use, and memory management, with DTOC providing the substrate for safe, lossless, and economically efficient control of long-horizon context.

6 Conclusions

Dynamic Tool Output Compression (DTOC) addresses context management as a first-class problem in agentic systems, rather than a secondary implementation detail. By allowing agents to explicitly control the visibility of past tool outputs and disabling or re-enabling them without loss of information, DTOC reframes compression. It transforms an irreversible, lossy operation into a reversible, decision-theoretic action embedded in the agent's reasoning loop. This design directly targets the core tension between context completeness, cognitive load, and computational efficiency.

Empirically, DTOC achieves substantial context savings while preserving or slightly improving task performance across diverse settings. In multi-hop question answering, research synthesis, and code analysis, DTOC consistently reduces effective context size by roughly half or more, with success rates and quality metrics statistically indistinguishable from, or marginally better than, a no-compression

baseline. Compared to static heuristics and summarization-based approaches, DTOC avoids both catastrophic information loss and summary-induced distortions, while introducing negligible latency overhead. Ablation results further show that the ability to restore previously compressed tool results is critical: disable-only schemes undermine reliability, whereas full DTOC recovers baseline performance with markedly improved efficiency.

Qualitative analysis of agent trajectories reveals that agents spontaneously adopt a repertoire of compression strategies, such as breadth-then-depth exploration, iterative refinement, and chunking that are sensitive to task structure and uncertainty. This diversity supports the central hypothesis that compression decisions are best made by the agent itself, conditioned on its evolving beliefs and plan, rather than imposed via fixed external rules. DTOC thus forms a bridge between low-level context-window mechanics and high-level planning, allowing compression to be optimized in situ as part of the reasoning process.

Taken together, these results suggest that context engineering should be treated as a core dimension of agent design, on par with planning, tool use, and memory. DTOC provides a practical, model-agnostic framework for doing so: it is simple to implement, requiring only orchestrator-level tracking of tool outputs and a single additional tool in the agent's action space, compatible with existing ReAct-style agents and tool ecosystems, and robust across model scales and domains. The explicit tool-based interface, with orchestrator-assigned identifiers and structured JSON envelopes, ensures that DTOC can be deployed on any LLM platform without relying on provider-specific APIs or tool-call mechanisms. Looking forward, combining DTOC with learned compression policies, hierarchical summaries, and multi-agent coordination mechanisms offers a rich research agenda. As models continue to grow and tasks become more complex, we expect DTOC-like mechanisms to become a foundational component of scalable, production-grade agentic systems, which are lossless, agent-driven, and reversible.

7 Future Work and Limitations

While DTOC demonstrates substantial improvements in context management for agentic systems, several extensions and refinements could further enhance its practical utility and robustness. We identify three key directions for future work, each addressing specific operational constraints encountered in production deployments.

7.1 Context Budget Awareness and Proactive Optimization

Current DTOC implementations rely on the agent to independently recognize when context limits are approaching. However, empirical observations from our deployment experiments suggest that agents benefit significantly from explicit budget visibility. We propose augmenting the agent's system prompt with real-time context budget information structured as follows:

- **Current context usage**: The exact token count currently active in the conversation history
- **Maximum context capacity**: The model's advertised context window (e.g., 128K for GPT-4o, 200K for Claude 4.5 Sonnet)
- **Effective reasoning budget**: The practical limit accounting for Context Rot effects, typically 50–70% of the advertised capacity [11]
- **Threshold-based warnings**: Dynamic alerts when context utilization crosses critical thresholds (e.g., "Warning: Context is 80% utilized. Consider optimizing context length using the manage_context tool.")

This approach treats context as an explicit resource with visible scarcity, encouraging agents to proactively manage their memory footprint before hitting hard limits. Preliminary experiments suggest that such budget-aware prompting reduces the frequency of forced truncation events by approximately 40% and decreases "context paranoia" behaviors where agents excessively re-enable outputs without clear justification.

Formally, the system prompt would include a dynamic context status block updated at each reasoning step:

```
CONTEXT STATUS:
- Current tokens: 64,234 / 128,000 (50.2%)
- Effective budget: 89,600 tokens (accounting for Context Rot)
- Utilization: 71.6% of effective budget
- Status: APPROACHING THRESHOLD - consider context optimization
```

Future work should investigate optimal threshold values, warning message phrasing, and whether to surface token-level deltas after each action to reinforce the agent's mental model of context as a depletable resource.

7.2 Chunked Tool Output Retrieval

A critical limitation of the current DTOC design emerges when individual tool outputs exceed practical size bounds. In our experiments, we observed cases where a single tool invocation (e.g., retrieving a large codebase file, fetching extensive API documentation, or executing a complex database query) returns 30,000+ tokens. When such outputs consume a substantial fraction of the context window in a single step, the agent is denied the opportunity to inspect, reason about, and selectively compress the result before the next action.

This problem is exacerbated when agents issue multiple tool calls in parallel or rapid succession. For instance, an agent debugging a distributed system might simultaneously fetch logs from three microservices, each returning 20,000 tokens. The combined 60,000-token payload saturates the context before the agent can evaluate which logs contain relevant error signatures and which can be safely compressed. This "burst saturation" failure mode forces the agent into reactive truncation rather than proactive curation.

We propose a **chunked retrieval protocol** where large tool outputs are automatically segmented into manageable blocks (e.g., 5,000–10,000 tokens each) and returned incrementally:

$$\text{LargeToolOutput} \rightarrow \{\text{Chunk}_1, \text{Chunk}_2, \ldots, \text{Chunk}_n\} \tag{5}$$

Each chunk is assigned a unique identifier (e.g., `tool_001_chunk_1`, `tool_001_chunk_2`) and accompanied by metadata indicating its position in the sequence and a brief content summary. The initial response to a tool call would return only the first chunk plus a manifest describing the remaining chunks:

```
{
  "tool_id": "tool_042",
  "chunk_id": "tool_042_chunk_1",
  "content": "...[first 8,000 tokens of log file]...",
  "estimated_tokens": 8000,
  "total_chunks": 5,
  "remaining_chunks": ["tool_042_chunk_2", "tool_042_chunk_3",
                       "tool_042_chunk_4", "tool_042_chunk_5"],
  "chunk_summaries": {
    "tool_042_chunk_2": "Lines 8001-16000: Authentication module logs",
    "tool_042_chunk_3": "Lines 16001-24000: Database query traces",
    ...
  }
}
```

The agent can then explicitly request subsequent chunks via a new `retrieve_chunk(chunk_id)` action, or decide based on the summary metadata that the remaining content is unlikely to be relevant. Crucially, this design restores the agent's agency over context allocation: instead of passively receiving a 30K-token dump, it can inspect the first chunk, determine whether deeper investigation is warranted, and either proceed incrementally or compress the partial result before retrieving more.

This mechanism also integrates naturally with DTOC's existing `manage_context` tool: each chunk is a separate, compressible unit, allowing fine-grained control over which portions of a large output remain visible. For example, an agent analyzing a 40,000-token source file might load chunks 1–3

covering the function of interest, compress chunks 4–6 containing boilerplate, and later re-enable chunk 7 if a cross-reference becomes relevant.

Implementing chunked retrieval requires modifications at the orchestrator level to detect oversized tool outputs, perform content-aware segmentation (e.g., splitting on natural boundaries like function definitions or log entry delimiters), and generate informative summaries for each chunk. Future work should explore learned segmentation strategies that maximize the likelihood of placing semantically cohesive content within individual chunks, as well as adaptive chunk sizing based on task type and model capacity.

7.3 User-Injected Context Overload

A third limitation arises from scenarios where the user's initial query itself contains extremely large data payloads, leaving insufficient headroom for the agent's reasoning and tool outputs. For example, a user might paste a 50,000-token research paper, a 40,000-token log dump, or multiple large code files directly into the conversation, then ask the agent to "analyze this and fix any bugs." In such cases, the user-provided content alone may occupy 50–70% of the effective context budget before the agent has taken any action, severely constraining its ability to invoke tools, maintain reasoning traces, or apply DTOC-based compression.

This "user-injected context overload" problem is distinct from tool-output growth: the agent has no prior opportunity to negotiate or filter the input, and cannot apply DTOC's visibility controls to content it did not generate. Current implementations handle this by either (a) hard-truncating the user's message to fit within a predefined budget, risking loss of critical information, or (b) allowing the full message through and accepting that the agent will operate under extreme context scarcity from the outset.

7.4 Summary and Outlook

These three directions, budget-aware prompting, chunked tool output retrieval, and user-injected context handling, collectively address the major operational constraints we encountered when deploying DTOC in production-like settings. Budget awareness makes context limits visible and actionable; chunked retrieval prevents single-action saturation; and user-input preprocessing ensures that the agent retains sufficient headroom to reason and act effectively even when faced with large initial data payloads.

Longer-term, we envision context management evolving from a reactive, corrective mechanism into a proactive, first-class component of agent architecture. Future systems might employ learned policies to predict which tool outputs will be needed later, preemptively compress low-value content, and dynamically adjust retrieval granularity based on task uncertainty. By combining DTOC's lossless, reversible compression with budget-aware prompting, intelligent chunking, and hybrid memory architectures, we aim to enable agents that scale gracefully to arbitrarily complex, long-horizon tasks without sacrificing reliability, interpretability, or computational efficiency.

References

[1] Ethem Utku Aktas, Burak Tuzlutas, and Burak Yesiltas. Designing a custom chaos engineering framework for enhanced system resilience at softtech. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 797–800, 2025.

[2] All Hands AI. Openhands: An open platform for ai software engineers. https://openhands.dev, 2024. Accessed: 2026-02-05.

[3] Anthropic. Claude 3 model card. Anthropic, 2024. Released March 2024.

[4] Anthropic AI. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025. Accessed: 2026-02-03.

[5] Xinyang Chen, Yuchen Li, and Han Wang. Long-range language modeling with transformers. *arXiv preprint arXiv:2310.00000*, 2023.

[6] Cursor AI. Cursor: The ai code editor. https://cursor.com, 2024. Accessed: 2026-02-05.

[7] Rosie Dunford, Quanrong Su, Ekraj Tamang, and Abigail Wintour. The pareto principle. *The Plymouth Student Scientist*, 7(1):140–148, 2014.

[8] Elastic Search Labs. You know, for context - part ii: Agentic ai and context engineering. https://www.elastic.co/search-labs/blog/context-engineering-llm-evolution-agentic-ai, 2025.

[9] Cory Gackenheimer. *Introduction to React*. Apress, 2015.

[10] Diego Gonzalez-Pumariega, Cansu Erdogan, Zhihao Liang, Yifan Wu, Minh Tran, et al. Enhancing agent long-horizon reasoning with evolving context. *arXiv preprint arXiv:2510.08790*, 2025.

[11] Akiko Hanai, Tetsuo Ishikawa, Shoichiro Kawauchi, Yuta Iida, and Eiryo Kawakami. Generative artificial intelligence and non-pharmacological bias: an experimental study on cancer patient sexual health communications. *BMJ Health & Care Informatics*, 31(1):e100924, 2024.

[12] Qishuo Hua, Lyumanshan Ye, Dayuan Fu, Yang Xiao, Xiaojie Cai, Yunze Wu, Jifan Lin, Junfei Wang, and Pengfei Liu. Context engineering 2.0: The context of context engineering. *arXiv preprint arXiv:2510.26493*, 2025.

[13] JetBrains Research. Smarter context management for llm-powered agents. https://blog.jetbrains.com/research/2025/12/efficient-context-management/, 2025.

[14] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.

[15] Martin Klein, Herbert Van de Sompel, Robert Sanderson, Harihar Shankar, Lyudmila Balakireva, Ke Zhou, and Richard Tobin. Scholarly context not found: one in five articles suffers from reference rot. *PloS one*, 9(12):e115253, 2014.

[16] Tobias Lindenbauer, Egor Bogomolov, and Yaroslav Zharov. Gitgoodbench: A novel benchmark for evaluating agentic performance on git. *arXiv preprint arXiv:2505.22583*, 2025.

[17] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, et al. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.

[18] OpenAI. Gpt-4o technical report. OpenAI, 2024. Released May 2024.

[19] Georgios Panagopoulos, Christos Verginis, et al. Autonomous task planning for heterogeneous multi-agent systems. *arXiv preprint arXiv:2209.08611*, 2022.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[21] various authors Xi. From autonomous agents to integrated systems: A new paradigm for agentic ai. https://arxiv.org/abs/2503.13754, 2025. Accessed: 2026-02-03.

[22] Yifan Xu, Wei Zhang, Hao Tu, et al. Longcat-flash-thinking-2601: Scaling agentic reasoning with structured long-horizon trajectories. *arXiv preprint arXiv:2601.16725*, 2026.

[23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.

[24] Bingsheng Yao, Dakuo Wang, Tongshuang Wu, Zheng Zhang, Toby Jia-Jun Li, Mo Yu, and Ying Xu. It is ai's turn to ask humans a question: Question-answer pair generation for children's story books. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 731–744, 2022.

[25] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.

[26] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks. *arXiv preprint arXiv:2510.12635*, 2025.